SF2935 Modern Methods of Statistical Learning
Project: Musical Classification

Group 011:
Vincent Hallberg
Ville Wassberg
Klara Zimmerman

October 2024

Problem statement

The goal of this project is to classify songs from Spotify based on key features such as danceability, energy, and loudness, in order to determine whether they would appeal to our teacher. That is, we aim to train an algorithm that analyzes the available features of a new song and predicts whether it should be recommended.

We have a dataset of 500 songs, each labeled with a binary value (0 or 1) indicating whether the teacher likes the song. To build and evaluate our models, we will split the dataset into training and test sets, using cross-validation on the training set and then trying it on the test set to assess the model performance.

For each model, we will tune the parameters involved to increase the performance. This is done using cross validation together with a discrete grid search to optimize the model. We must also take into account the size of the training- vs test sets, as well as the number of folds in the cross validation.

We used the programming language Python to create these models, and imported the libraries pandas, sklearn, numpy, torch, lightgbm and xgboost.

Data exploration

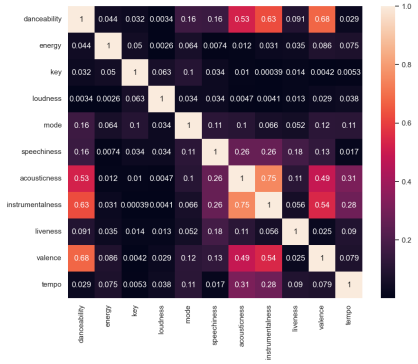
We begin by plotting the data to investigate relations and outliers visually in the data. By plotting the features pairwise against each other, relationships as well as distributions can be spotted. In Figure 1a one can spot clear outliers in several features. By interpolating points using scikit's method *imputer*, we replaced outliers in the 99^{th} percentile, of which the resulting data set can be seen in the right Figure 1b, where the features also have been scaled/normalised. Now the relations between features are more prominent, for instance there seems to be a quadratic relationship between energy and loudness and between loudness and acousticness, and we can see there are not clearly known distributions except perhaps some gamma distributions.

(a) Distribution and pair plots of features with clear outliers.

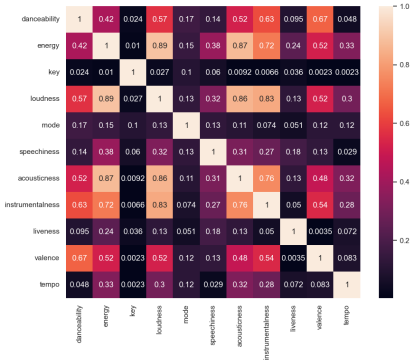
(b) Distribution and pair plots with clear outliers replaced/interpolated.

Figure 1: Pairwise feature plots, with and without outliers

When we replace the outliers, however, the linear relationships between the features become more prominent which can be seen in the difference between Figure 2a and 2b below.



(a) Covariance plot with outliers. Some features highly collinear.



(b) Covariance plot without clear outliers. As seen, features became even more collinear.

Figure 2: Covariance plots, with and without outliers

By manually handle these relations through transformations and removal was not beneficial for our models in terms of prediction accuracy of unseen data, and the linear relationships that showed when the outlier was replaced or removed made the tested models predict more poorly. Also, some models such as tree based methods are not sensitive to outliers nor linear relationships. Therefore, we chose to continue with the original data set, and instead tune hyper parame-

ters for regularisation techniques such as ridge regression, principal component analysis and Lasso, to the different models in order to avoid these adversities.

Method 1: Tree based methods

Tree based methods use decision trees to classify inputs by recursively splitting the data based on features. Each node in a decision tree represents a feature, each branch represents a decision rule, and each leaf represents a classification. In our case, the song features are the nodes and the classes (0 and 1) are the leaves.

We tested two different tree-based classification methods: Light Gradient Boost and X Gradient Boost. We used the pre-existing methods LGBMClassifier() from the lightgbm library and XGBClassifier() from the xgboost library, respectively.

Light Gradient Boost is a gradient boosting method that sequentially builds trees, where each tree tries to correct the errors from the previous tree. At each step, the gradient of the loss function is calculated, and based on this, new trees are formed. Instead of growing trees level by level, this method grows by leaves, splitting the leaf that minimizes the loss. This leads to deep trees that are often very accurate.

X Gradient Boost is also a gradient boosting method that sequentially builds trees, where each tree tries to correct the errors from the previous tree. The strategy for growing trees is however level-wise, meaning it is a depth-first method. This method tends to be slower than Light Gradient Boost, however it is less prone to over-fitting on small data sets.

The pre-existing python method we used for these two tree methods supports both L1 and L2 regularization terms. We tuned the values of the λ-factors, i.e. the strength of these regularizations, in a grid search. Other parameters we tuned in the grid search were the depth of the trees, the number of estimators, the number of leaves, the learning rate, the minimum data in each leaf and subsample which is a type of stochastic gradient descent randomly sampling a fraction of the data to use for each tree. Also, a parameter called minimum child weight was tuned, which lets one choose a threshold for the sum of the weights on each node of a tree. The optimal values in our test run along with the accuracy on the test set are shown in Table 1.

Hyperparameter/Function	**Value XGB**	**Value LGB**
Max Depth	12	10
Estimators	500	500
Subsample	0.6	0.8
Learning Rate	0.1	0.01
λ for L1-regularization	0	0
λ for L2-regularization	1	0.5
Test Set Accuracy	0.8218	0.8416

Table 1: Parameters and test results of the XGB and LGBM

Although the Light Gradient Boost method resulted in slightly higher test set accuracy than the X Gradient Boost method, we decided to focus on XGB in the following model comparisons. As we have a fairly small data set (500 data points) we need a model that is less prone to over-fitting.

Method 2: Support Vector Machines

Support Vector Machines (SVMs) classify data points by finding a hyperplane that best separates the two classes. The optimal hyperplane is the one that maximizes the margin between the closest data points from each class, also known as support vectors. In cases where the data is not linearly separable, SVMs use a kernel function to implicitly map the data to a higher-dimensional space, where a separating hyperplane can be found. In python this was done using the sklearn classifier SVC().

In our dataset, the number of features (dimensions) is 11, corresponding to the song attributes. The kernels we tested were the linear kernel (no mapping to a higher dimension), polynomial, Gaussian (RBF), and sigmoid kernels.

After evaluating the model using each kernel and cross-validating the training set, we selected the Gaussian (RBF) kernel, which is defined as:

$$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2\right),$$

where γ controls the influence of a single data point on the decision boundary, thereby affecting the smoothness of the boundary. A smaller γ creates a smoother decision boundary, while a larger γ leads to more complexity and flexibility in fitting the training data.

We also tuned the penalty C, which is a regularization parameter to prevent overfitting. It is essentially a Lagrange multiplier/relaxation, where a small C will allow some misclassifications making the model simpler, hence possibly increasing the margin. Since SVMs are sensitive to outliers and collinearity, we

first removed an obvious outlier as seen in Figure 1. However, when that was done the multicollinearity of the data set became more apparent. To resolve this issue, transforming features, using principal component analysis (PCA) and/or Lasso and ridge regression are useful methods. We found that manually transforming and/or remove features did not have a positive effect on the accuracy of the model. In fact, simply removing the extreme outlier made the model perform worse. Therefore, PCA was used with the hyperparameter 0.99 indicating we want to keep 99% of the variance, i.e. increasing our bias slightly.

Our tuned Support Vector Machine model, along with the performance accuracy on the test set, is presented in Table 2.

Hyperparameter/Function	**Value**
Kernel	Gaussian
γ	0.1
C	1.0
PCA	0.99
Test Set Accuracy	0.79208

Table 2: Parameters and test results of our Support Vector Machine

Method 3: Neural Networks

Neural networks are made up of layers of interconnected nodes, where each connection has an associated weight. These weights determine the importance of the inputs as they pass between layers, and the goal is to find the optimal weights for our model. In our case, the input will consists of the song features, and we want to find the weights that output the correct classification of the song.

Training a neural network involves using a backpropagation algorithm to adjust the weights in the network. This is done by first running each input through the network and comparing the prediction to the true label (classification). To determine the error we use binary cross entropy, as we only have two classes.

We then use a pre-existing optimizer method to update the weights. To find the most appropriate optimizer, we tested three methods: Stochastic Gradient Descent (SGD), Adaptive Moment Estimation (Adam), and AdamW. While SGD updates the weights by following the gradients, Adam and AdamW use adaptive learning rates for each parameter, adjusting them based on the mean and variance of the gradients. The main difference between the later two is that AdamW applies L2 regularization separately from the gradient update.

To find the best optimizer, as well as tune other parameters of the model, we performed a randomized grid search. The parameters we tuned were learning

rate, activation function, number of hidden layers, and number of epochs (how many times the dataset is passed through the model during training). Each time a new combination was tested, we performed a cross validation on our training data set, and chose the parameters with the best result. The number of hidden layers were only tested up to five, and the number of neurons chosen for each layer was not tuned much, although, changing these did not clearly affect performance.

Finally, we evaluated our model by running it on our test set. The result of this along with our optimal parameters is presented in Table 3.

Hyperparameter/Function	**Value**
Optimizer	Adam
Learning Rate	0.001
Activation Function	ReLU
Hidden Layers	2: (150, 50)
Number of Epochs	50
Test Set Accuracy	0.7723

Table 3: Parameters and test results of the Neural Network

Finding optimal hyperparameters

Choosing the optimal hyperparameters for different models is often a challenge, as many models include continuous hyperparameters, making it infeasible to explore the entire parameter space. To evaluate different hyperparameters in the models we used cross-validation grirdsearch and cross-validation randomized grid search.

Cross-validation is a standard practice to ensure robustness of a model. This is done by splitting the data into into k folds - training on k-1 of the folds, and testing it on the last fold. We used stratified k-fold to make sure we had the same distributions of classes in each fold. Stratified k-fold is especially useful when there is an imbalance of classes in the dataset, which can lead to inaccurate representations of the real distributions in the different folds. In our case however, the division of the two classes was very balanced, thus this method may not have been necessary.

To actually tune the hyperparameters, we initially used cross-validation grid search to systematically explore combinations of hyperparameters within specified ranges for each model. This approach allowed us to test predefined values across multiple hyperparameters to identify settings that yielded the best performance. However, due to the continuous nature of many hyperparameters, it is often impractical to test every possible value. Instead, grid search is typically used in an iterative process where we start with a broader range of values to

get a rough idea of which ranges seem promising.

Once initial grid search results indicated general parameter regions that performed well, we could narrow down these ranges and rerun the grid search with more finely tuned increments. This stepwise refinement helped us identify an optimal or near-optimal range without the need to exhaustively test the entire parameter space. By progressively refining the search space, grid search helps focus on high-performing parameters while remaining computationally feasible.

After identifying a reasonable range with grid search, we also applied cross-validation randomized grid search to further optimize the model. Unlike grid search, which tests all possible combinations in a defined grid, randomized grid search randomly samples parameter combinations within the specified ranges. This method is particularly useful for exploring larger parameter spaces because it allows a broader search in fewer iterations, helping to capture potential high-performing combinations. In this way, randomized search complements grid search by casting a wider net and can often reach a good enough set of hyperparameters in less time, especially when working with continuous or expansive hyperparameter ranges.

This combination of grid search and randomized search allowed us to efficiently and effectively explore the hyperparameter space, yielding a well-tuned model suited to the dataset and ensuring both robust performance and computational feasibility.

Comparing methods

One way to compare the chosen methods is to check their generalization and performance over different training sizes of the data. As seen in Figure 3 the XGB method has much less variation between the splits in the cross-validation, which indicates that it is a good model.

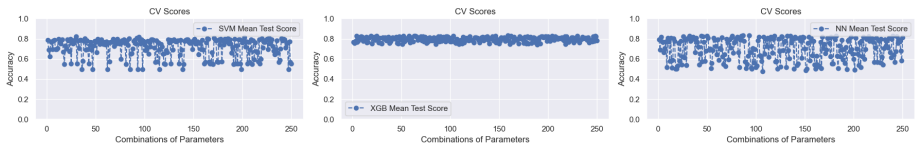


Figure 3: Score/accuracy of each cross validation performed by the methods.

Figure 4 shows the learning curves of each method plotted from their cross-validated results on different sizes of the data. As we can see the performance of the methods does not differ much on the test data, however, XGB seems to be slightly more accurate than the others. The curves of XGB on training data and CV does not seem to converge towards each other, indicating the XGB

method might be over-fitted to the data. However, we selected the XGB model to go into production since it has repeatedly performed better on unseen data than the others, with a test set accuracy of 0.82.

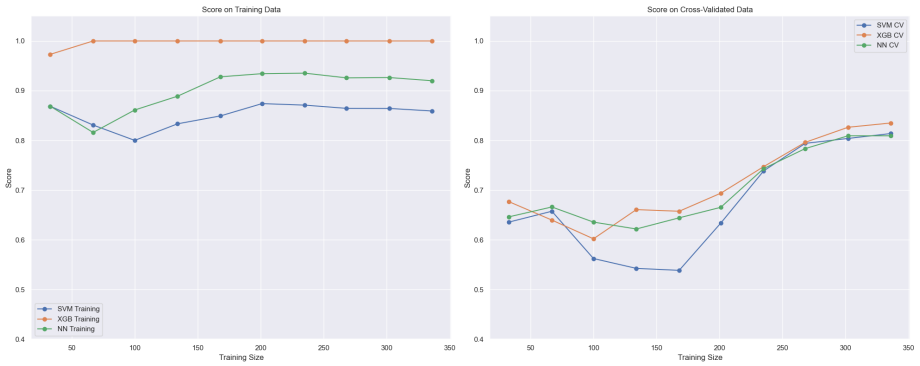


Figure 4: Learning curves plotted of methods using 3-folded cross validation.

Conclusion

In summary, we have trained and fitted three different classes of methods. Namely, tree based methods, support vector machines and artificial neural networks. The performance of the methods was evaluated by mainly comparing prediction accuracy on unseen data. Finally, the chosen method to use to classify the given test data is **X Gradient Boost** due to its consistency and robustness in cross validation and predictions of unseen data.